

P R O J E T O

ORBITAL TRUST

GS · 2026 · CIBERSEGURANÇA 1

Módulo de Cibersegurança

Identificação de riscos · Controles de segurança · Implementação prática

SEGURANÇA · ORBITAL · CIBERNÉTICA · APLICADA · AO ESPAÇO

// IDENTIFICAÇÃO

Equipe e Contexto

01

NOME	RM	RESPONSABILIDADE NA GS
Victor Dias	RM558017	Cibersegurança + SOA / WebServices
Gustavo Ruiz Vieira Paulino	RM554779	C# (POO / OOP) + Machine Learning
Fernando Luiz Silva Antonio	RM555201	Mobile (React Native) + IoT
Thomas Reichmann	RM554812	Sistemas Operacionais (OS)
Guilherme Abe	RM554743	Qualidade de Software (QA)

PROJETO	DISCIPLINA	PROFESSOR	ENTREGA
Orbital Trust	Cibersegurança 1	Prof. MSc. Oerton Fernandes	Junho / 2026

Sobre o sistema

O Orbital Trust é um MVP que transforma imagens orbitais abertas (Sentinel-2, Landsat e tiles públicos) em alertas ambientais confiáveis sobre queimadas, solo exposto, água, vegetação e baixa visibilidade. A arquitetura é desacoplada em três camadas: o **IoT/CV** (Python + OpenCV) carrega frames, extrai métricas de qualidade e mudança e gera um payload JSON padronizado; a **API/ML** (FastAPI) refina o risco ambiental e emite recomendações; e o **App Mobile** (React Native/Expo) exibe os alertas. A confiança da análise sustenta o índice de confiabilidade do alerta.

O problema não é a falta de dados espaciais — é transformar esses dados em decisões confiáveis.

Este documento aplica STRIDE sobre os cinco componentes da arquitetura, define controles de segurança com diagrama e documenta a implementação prática da equipe — a criptografia AES-256-GCM da coordenada do sensor, entregue no repositório `orbital-trust-sec`.

// SEÇÃO 01

1. Ativos Críticos

Cinco ativos mapeados na arquitetura. A criticidade reflete o impacto operacional caso o ativo seja comprometido:

#	ATIVO	DESCRIÇÃO	CRITICIDADE
1	API de Análise	Backend FastAPI (Python) que recebe o payload JSON do IoT, refina o risco ambiental e devolve o AlertResponse ao app. Ponto central de toda comunicação.	ALTA
2	Banco de Dados	Persistência (PostgreSQL, em Docker) do histórico de leituras e alertas. Guarda o campo Coordenada — sempre cifrado com AES-256-GCM. Alvo do controle implementado neste módulo.	ALTA
3	Nó IoT / CV	Pipeline Python + OpenCV que processa frames orbitais (Sentinel-2, Landsat), extrai métricas de qualidade e mudança e gera o payload JSON.	ALTA
4	Camada de ML	Classificação de risco ambiental (ml/predict) que refina o score do payload e gera a recomendação operacional.	MÉDIA
5	App Mobile	App React Native (Expo, TypeScript) que exibe alertas, histórico e métricas de confiança. Canal de acesso dos usuários finais.	MÉDIA

// SEÇÃO 02

2. Análise de Ameaças — STRIDE

Metodologia STRIDE aplicada sobre os cinco ativos. Listamos apenas ameaças com impacto real no contexto do Orbital Trust:

ATIVO	CATEGORIA	AMEAÇA	IMPACTO
API de Análise	Spoofing	Dispositivo não autenticado envia payload fabricado como sensor legítimo.	Sistema gera alerta falso durante evento crítico.
API de Análise	Tampering	Dados em trânsito alterados sem TLS — risk_level modificado de alto para baixo.	Falha na resposta a desastre ambiental.
API de Análise	Repudiation	Sem log de auditoria, impossível rastrear qual fonte gerou alerta incorreto.	Investigação pós-incidente inviabilizada.
API de Análise	Info. Discl.	Endpoint sem autenticação expõe coordenadas de ativos e histórico completo.	Vazamento de localização de infraestrutura crítica.
API de Análise	DoS	Flood de requisições paralisa a API durante evento ambiental real.	Alertas legítimos não chegam aos usuários.
API de Análise	Priv. Escal.	Token de sensor_node usado em rotas admin por falta de validação de role.	Acesso indevido a configurações completas.
Banco de Dados	Tampering	Injeção SQL em endpoint de consulta altera histórico de leituras ambientais.	Corrupção dos dados que alimentam o ML.
Banco de Dados	Info. Discl.	Banco exposto com credenciais padrão — acesso ao histórico completo.	Vazamento de histórico ambiental e localizações.
Banco de Dados	DoS	Queries maliciosas travam o banco, parando o processamento de alertas.	Sistema cessa de emitir alertas em tempo real.
Nó IoT / CV	Spoofing	Frames adulterados injetados no pipeline enganam o modelo de CV.	ICO sobe com dado falso de ground truth.
Nó IoT / CV	Tampering	Pipeline comprometido via acesso ao ambiente de execução.	Payload JSON manipulado na origem, sem detecção.
Nó IoT / CV	Priv. Escal.	Ambiente sem hardening usado como pivot para atacar a rede interna.	Movimento lateral para API, banco e demais módulos.
Camada de ML	Tampering	Arquivo do modelo substituído por versão envenenada com decision boundary alterado.	Classificações erradas — falsos negativos em crises.
Camada de ML	Info. Discl.	Acesso ao modelo permite inferir dados de treinamento por model inversion.	Vazamento indireto de dados históricos sensíveis.
App Mobile	Spoofing	Sessão sequestrada por token sem expiração ou session fixation.	Acesso indevido ao app por terceiros.
App Mobile	Info. Discl.	XSS ou interceptação exfiltra tokens de sessão de analistas.	Comprometimento de credenciais com acesso a dados.

// SEÇÃO 03

3. Controles de Segurança

Cinco controles escolhidos pela aplicabilidade direta na arquitetura do Orbital Trust, cada um mitigando ameaças reais identificadas na análise STRIDE:

CONTROLE	TIPO	APLICA EM	MITIGA
TLS/HTTPS obrigatório	Preventivo	API de Análise — toda comunicação	Tampering em trânsito · Spoofing de canal
Autenticação + Roles IAM	Preventivo	API — sensor_node, analyst, admin	Elevation of Privilege · Spoofing de identidade
Criptografia AES-256-GCM	Preventivo	Coordenada no banco PostgreSQL (implementado)	Information Disclosure em acesso indevido · Tampering
Monitoramento de logs	Detectivo	API + persistência — requisições e predições	Repudiation · Padrões anômalos
Autenticação mTLS para IoT	Preventivo	Nó IoT/CV — certificado bilateral	Spoofing de sensor · Tampering na origem

Diagrama — Distribuição dos controles pela arquitetura



// SEÇÃO 04

4. Implementação Prática

Criptografia AES-256-GCM no campo de coordenada (C#)

Implementamos AES-256-GCM para proteger a `coordenada` do sensor — o dado mais sensível do payload, pois revela a localização exata da infraestrutura monitorada. A escolha foi deliberada: entregar um controle de segurança real e auditável. A gente quase foi de AES-CBC no início — mas depois de ler sobre padding oracle mudamos pra GCM, que tem autenticação embutida (AEAD) e elimina uma classe inteira de ataques sem custo adicional de performance.

A implementação é em **C# / .NET**: o `AesGcm` nativo entrega criptografia autenticada (AEAD) sem dependências externas, com código enxuto e auditável. O serviço expõe `Encrypt` / `Decrypt` sobre um contrato simples (string → string em Base64), de modo que qualquer camada da solução — inclusive a API em Python, via serviço ou porta dedicada — pode consumir o controle sem acoplamento de linguagem. O `CryptoService` está no repositório `orbital-trust-sec`, com testes xUnit cobrindo roundtrip e três vetores de adulteração.

```
CryptoService.cs C# · AES-256-GCM

// System.Security.Cryptography + System.Text
namespace OrbitalTrust.Security;

public static class CryptoService
{
    // Chave 256-bit via env var ORBITAL_TRUST_KEY (Base64 de 32 bytes).
    // Fallback demo apenas para a entrega; em produção, Azure Key Vault.
    private const string _demoKey = "K7gNU3sdo+OL0wNhqoVWhr3g6slxYv72o1/pe/Unols=";
    private static readonly byte[] _key = Convert.FromBase64String(
        Environment.GetEnvironmentVariable("ORBITAL_TRUST_KEY") ?? _demoKey);

    public static string Encrypt(string plaintext)
    {
        using var aes = new AesGcm(_key, 16); // tag 128 bits
        var nonce = new byte[12]; // nonce 96 bits - padrão GCM
        var tag = new byte[16];
        var input = Encoding.UTF8.GetBytes(plaintext);
        var cipher = new byte[input.Length];
        RandomNumberGenerator.Fill(nonce);
        aes.Encrypt(nonce, input, cipher, tag);
        // Serializa: nonce(12) | tag(16) | ciphertext → Base64
        return Convert.ToBase64String(
            nonce.Concat(tag).Concat(cipher).ToArray());
    }

    public static string Decrypt(string b64)
```

Coordenada cifrada em repouso — banco real (PostgreSQL · Docker)

O controle não é hipotético: um `docker compose up` sobe um PostgreSQL e o app `DbSeed` (reusando o mesmo `CryptoService`) grava a coordenada **já cifrada**. Um `SELECT` cru na tabela retorna apenas Base64 — mesmo com acesso total ao banco, a localização exata do sensor permanece protegida. É a mitigação de **Information Disclosure** em *data at rest*, tangível.

Evidências no repositório: `output.txt` + `print_execucao.png` (três cenários do `CryptoService`) e `output-banco.txt` + `print_banco.png` (gravação no Postgres + `SELECT` cru). Testes xUnit em `tests/` cobrindo roundtrip, unicidade e três vetores de adulteração.

// SEÇÃO 05

5. Integração com GS e ODS

O Módulo de Cibersegurança está integrado ao Orbital Trust e conectado a dois Objetivos de Desenvolvimento Sustentável da ONU:

ODS	CONEXÃO COM O ORBITAL TRUST
ODS 13 — Ação Climática	O sistema monitora eventos ambientais em tempo real. Segurança garante que alertas sobre queimadas, enchentes e seca chegam íntegros a quem precisa agir. Um alerta adulterado ou suprimido tem impacto direto na resposta a desastres climáticos — comprometer a integridade do dado é comprometer a ação ambiental.
ODS 9 — Inovação e Infraestrutura	A solução usa infraestrutura espacial e IA aplicada a problema real. Cibersegurança garante a confiabilidade: pipeline de ML ou API comprometidos invalidam toda a proposta de valor do Orbital Trust — um ICO calculado sobre dados adulterados não é confiável.

// SEÇÃO 06

6. Repositório e Reprodução

Código, testes e evidências versionados no GitHub:

ITEM	LOCALIZAÇÃO
Repositório	github.com/vitordpacifico/orbital-trust-sec
Implementação	src/CryptoService/CryptoService.cs
Demonstração	src/CryptoService/Program.cs (3 cenários)
Banco (Docker)	docker-compose.yml · db/init.sql · src/DbSeed/
Testes (xUnit)	tests/CryptoService.Tests/
Evidências	evidencias/ — output.txt, print_execucao.png, output-banco.txt, print_banco.png

```
terminal · como executar bash · .NET

# Demonstração standalone (3 cenários)
dotnet run --project src/CryptoService

# Banco real: sobe Postgres e grava coordenada cifrada
docker compose up -d && dotnet run --project src/DbSeed/

# Testes
dotnet test tests/CryptoService.Tests
```

// REFERÊNCIAS

Referências

- › STRIDE Threat Model — Microsoft Secure Development Lifecycle. microsoft.com/securityengineering/sdl/threatmodeling
- › NIST SP 800-38D — Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. NIST, 2007.
- › OWASP Top 10 2021 — A03: Injection · A07: Authentication Failures. owasp.org/Top10
- › NASA FIRMS — Fire Information for Resource Management System. firms.modaps.eosdis.nasa.gov
- › MITRE ATT&CK for ICS — Tactics and Techniques for ICS. attack.mitre.org/matrices/ics